LIMKOKWING UNIVERSITY OF CREATIVE TECHNOLOGY
ESWATINI

WEEK 1 : LECTURE 1 - introduction

Bachelor's Degree in Information Technology
BITSD4111, SOFTWARE DESIGN
Trimester: 06
From **March 2024** to **June 2024**

OBJECTIVES

- Introduction
- Aims and Outcomes.
- Teaching and Assessment.
- Recommended Reading.
- Software Development Terminology.
- Design Objectives and Design Problems.
- Why Do Things Go Wrong?
- Project Life Cycles.
- What is object-orientation?
- Why object-orientation?
- UML Background.
- Models and Diagrams.
- UML Tools.
- Objects concepts.
- Summary

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

INTRODUCTION

- Software Design is the process of transforming user requirements into a suitable form, which helps the programmer in software coding and implementation.

- During the software design phase, the design document is produced, based on the customer requirements as documented in the Software Requirements Specification (SRS) document.

- Hence, this phase aims to transform the SRS document into a design document.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

INTRODUCTION

- The following items are designed and documented during the design phase:
  - Different modules are required.
  - Control relationships among modules.
  - Interface among different modules.
  - Data structure among the different modules.
  - Algorithms are required to be implemented among the individual modules.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

AIMS AND OUTCOMES

- Upon completion of this module, participants should know and understand:
  - Software requirements and its role in determining the architecture design
  - Object Oriented Architecture (OO Analysis and Design)
  - Design Principles and their role in leading to sustainable and dependable system
  - The domain of application of different architectural styles
  - Model Driven Software Development
  - Software Product Families and Feature Modeling

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

TEACHING AND ASSESSMENT

- Lecture notes/slides for each topic will be provided
- Assessment will be as follows:

ASSIGNMENT DESCRIPTION	ISSUE DATE	DUE DATE	%
Group Project Document	Week 2	Week 12	15%
Class Test #1	Week 4	Week 4	15%
Class Test #2	Week 8	Week 8	15%
Final Examination	Week 13	Week 13	50%
TOTAL			**100%**

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

RECOMMENDED READING

Prescribed texts books (Main Reading)

- T.C.Lethbridge and R.Laganiere, Object-Oriented Software Engineering, 2nd Edition: Practical Software Development using UML and Java, McGraw-Hill, 2005

Additional Books

- Bennet, Simon, Object Oriented System Analysis & Design using UML, McGraw-Hill, 2002
- Siau, Keng; Halpin, Terry, Unified Modeling Language: Systems Analysis, Design and Development Issues, Idea Group Publishing, 2001
- Favre, Liliana, UML AND THE UNIFIED PROCESS, Idea Group Inc., 2003
- Yang, Hongji, Software Evolution with UML and XML, Idea Group Publishing, 2004
- Eric J. Braude. Software Design: From Programming to Architecture, John Wiley & Sons, 2004, ISBN:0-471-42920-1.
- Martin Fowler. Refactoring, Improving the Design of Existing Code, Addison Wesley, 1999.
- Pressman, R.S. Software Engineering: A Practitioner's Approach, 6th Ed., McGraw-Hill Companies, Inc. 2005.
- Somerville, Ian. Software Engineering, 6th Ed., Addison-Wesley, 2001.
- David Budgen, Software Design, Pearson, Addison Wesley, second edition 2003

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

SOFTWARE DEVELOPMENT TERMINOLOGY

- Software Development
- Software Engineering
- Software Design

- "software engineering" encompasses the entire process of designing, building, and maintaining software systems,
- “software design" focuses specifically on the planning and conceptualization of the software architecture
- “software development" refers to the act of writing code to implement that design and bring the software to life

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

SOFTWARE DEVELOPMENT TERMINOLOGY

- Abstraction (Hide Irrelevant data): Abstraction simply means to hide the details to reduce complexity and increase efficiency or quality.
  - ✓ Different levels of Abstraction are necessary & must be applied at each stage of the design process so that any error that is present can be removed to increase the efficiency of the software solution & to refine the software solution.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

SOFTWARE DEVELOPMENT TERMINOLOGY

- Modularity (subdivide the system): Modularity simply means dividing the system or project into smaller parts to reduce the complexity of the system or project.
  - ✓ In the same way, modularity in design means subdividing a system into smaller parts so that these parts can be created independently and then use these parts in different systems to perform different functions..

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

SOFTWARE DEVELOPMENT TERMINOLOGY

- Architecture (design a structure of something): Architecture simply means a technique to design a structure of something. Architecture in designing software is a concept that focuses on various elements and the data of the structure.
  - ✓ These components interact with each other and use the data of the structure in architecture.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

SOFTWARE DEVELOPMENT TERMINOLOGY

- Refinement (removes impurities): Refinement simply means to refine something to remove any impurities if present and increase the quality.
  - ✓ The refinement concept of software design is a process of developing or presenting the software or system in a detailed manner which means elaborating a system or software.
  - ✓ Refinement is very necessary to find out any error if present and then to reduce it.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

SOFTWARE DEVELOPMENT TERMINOLOGY

- Pattern (a Repeated form): A pattern simply means a repeated form or design in which the same shape is repeated several times to form a pattern.
  - ✓ The pattern in the design process means the repetition of a solution to a common recurring problem within a certain context.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

SOFTWARE DEVELOPMENT TERMINOLOGY

- Information Hiding (Hide the Information): Information hiding simply means to hide the information so that it cannot be accessed by an unwanted party.
  - ✓ In software design, information hiding is achieved by designing the modules in a manner that the information gathered or contained in one module is hidden and can't be accessed by any other modules.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

SOFTWARE DEVELOPMENT TERMINOLOGY

- Refactoring (Reconstruct something): Refactoring simply means reconstructing something in such a way that it does not affect the behavior of any other features.
  - ✓ Refactoring in software design means reconstructing the design to reduce complexity and simplify it without impacting the behavior or its functions.
  - ✓ "the process of changing a software system in a way that it won't impact the behavior of the design and improves the internal structure".

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

DESIGN OBJECTIVES

- Objectives of Software Design
  - Correctness: A good design should be correct i.e., it should correctly implement all the functionalities of the system.
  - Efficiency: A good software design should address the resources, time, and cost optimization issues.
  - Understandability: A good design should be easily understandable, it should be modular, and all the modules are arranged in layers.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

DESIGN OBJECTIVES

- Objectives of Software Design
  - Flexibility: A good software design should have the ability to adapt and accommodate changes easily.
    - ✓ It includes designing the software in a way, that allows for modifications, enhancements, and scalability without requiring significant rework or causing major disruptions to the existing functionality.
  - Completeness: The design should have all the components like data structures, modules, external interfaces, etc.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

DESIGN OBJECTIVES

- Objectives of Software Design
  - Maintainability: A good software design aims to create a system that is easy to understand, modify, & maintain over time.
    - ✓ This involves using modular & well-structured design principles e.g. employing appropriate naming conventions and providing clear documentation.
    - ✓ Maintainability in Software and design also enables developers to fix bugs, enhance features, and adapt the software to changing requirements without excessive effort or introducing new issues.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

DESIGN PROBLEMS

- Ever-Changing Software Requirements
  - ✓ We know this thing from the beginning of software requirements do keep on changing but keeping a close eye on them is extremely important.
  - ✓ Unclear requirements often result in unwanted mishaps and misfortunes.
  - ✓ In addition, not understanding the needs right also leads to a huge waste of time and cost.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

DESIGN PROBLEMS

- Software Bugs and Broken Code
  - ✓ Software testing was once considered an afterthought.
  - ✓ This is one of the major problems created by software developers all across the globe.
  - ✓ Underestimating the formal quality assurance process during software product development is the biggest issue here.
  - ✓ As a solution, software development and software testing must go hand-in-hand, as this will surely lead to a successful launch

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

DESIGN PROBLEMS

- Lack of Collaborative Communication
  - ✓ Communication has been one of the major issues, especially in the information technology industry.
  - ✓ They are coders or programmers, most of them are geeks who might not be able to communicate well.
  - ✓ Effective communication is the key here and if not done properly then It might lead to delay of the software solution or even cost you unnecessary money.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

DESIGN PROBLEMS

- Confidentiality of Business Information
  - ✓ Understanding the confidentiality of business deals with the part where you know which information to share and which shouldn't be shared.
  - ✓ It might possibly make you feel overwhelmed.
  - ✓ Thus, you need to be extra cautious while dealing with third party companies and sharing your financial information to them. But there is a solution to this oversharing of information.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

WHY DO THINGS GO WRONG?

- Causes of software project failures

Type of failure	Reason for failure	Comment
Quality problems	The wrong problem is addressed	System conflicts with business strategy
	Wider influences are neglected	Organization culture may be ignored
	Analysis[2] is carried out incorrectly	Team is poorly skilled, or inadequately resourced
	The project is undertaken for the wrong reason	Technology pull or political push
Productivity problems	Users change their minds	
	External events change the environment	New legislation
	Implementation is not feasible	May not be known until the project has started
	Poor project control	Inexperienced project manager

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

WHY DO THINGS GO WRONG?

- Software projects generally fail on grounds of either unacceptable quality or poor productivity.
- In either case, the proposed system may never be delivered, it may be rejected by its users or it may be accepted yet still fail to meet its requirements.
- These 2 categories are known as 'ideal types'.
- They are meant to explain what is found in reality, but that does not imply that any real example will precisely, in all its details, match any one category.
- Real projects are complex, and their problems can seldom be reduced to one single cause

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

WHY DO THINGS GO WRONG?

- Quality problems
  - ✓ One of the most widespread definitions of the quality of a product is in terms of its 'fitness for purpose'
  - ✓ In order to apply this to the quality of a computer system, clearly it is necessary to know first for what purpose the system is intended and second, how to measure its fitness.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

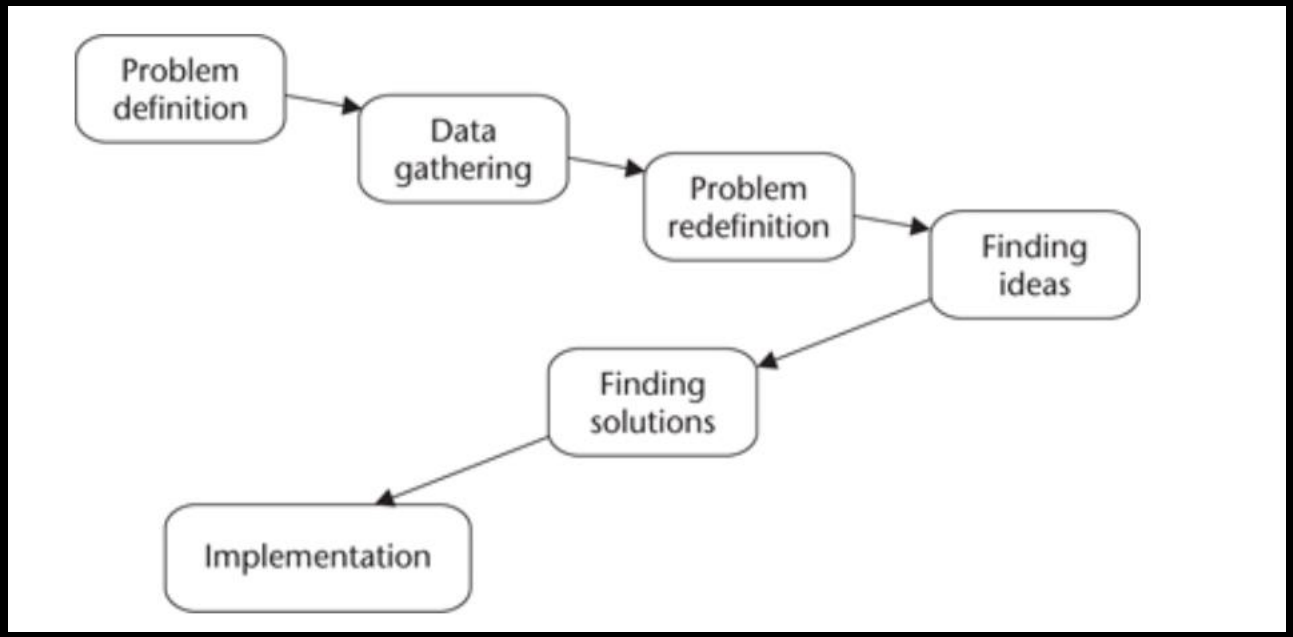
WHY DO THINGS GO WRONG?

- Productivity problems
  - ✓ Productivity problems relate to the rate of progress of a project, and the resources (including time and money) that it consumes along the way.
  - ✓ If quality is the first concern of users and clients, then productivity is their other vital concern.
  - ✓ The questions that are likely to be asked about productivity are as follows.
    - ➢ Will the product be delivered?
    - ➢ Will it be delivered in time to be useful?
    - ➢ Will it be affordable?

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

PROJECT LIFE CYCLES

- There are many different approaches but most utilize, in some form or other, a general problem-solving approach shown below:



ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

PROJECT LIFE CYCLES

- The phases Data gathering & Problem redefinition are concerned with understanding what the problem is about
- the Finding ideas phase attempts to identify ideas that help us to understand more about the nature of the problem and possible solutions.
- Finding solutions is concerned with providing a solution to the problem & Implementation puts the solution into practice.
- This approach to problem solving divides a task into subtasks, each with a particular focus & objective.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

PROJECT LIFE CYCLES

- The software development process may be subdivided simply into three main tasks:
  - ✓ understanding the problem
  - ✓ choosing and designing a solution
  - ✓ building the solution.
- subdividing a software project includes:
  - ✓ an analysis activity that identifies what the system should do
  - ✓ a design activity that determines how best to do it
  - ✓ A construction activity that builds the system according to the design

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

PROJECT LIFE CYCLES

- The Waterfall Lifecycle Model
  - The Waterfall Model is a linear application development model that uses rigid phases
  - When one phase ends, the next begins.
  - Steps occur in sequence, and, if unmodified, the model does not allow developers to go back to previous steps (hence "waterfall": Once water falls down, it cannot go back up).
  - It is one of the first software development methodologies

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Project Life Cycles

- The Waterfall Lifecycle Model
  - One of the benefits is that the phases have explicitly defined products or deliverables outlined below.

Phase	Output deliverables
System engineering	High-level architectural specification
Requirements analysis	Requirements specification Functional specification Acceptance test specification
Design	Software architecture specification System test specification Design specification Subsystem test specification Unit test specification
Construction	Program code
Testing	Unit test report Subsystem test report System test report Acceptance test report Completed system
Installation	Installed system
Maintenance	Change requests Change request report

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Project Life Cycles

- The Waterfall Lifecycle Model
  - These products can be used to monitor productivity and the quality of the activity performed.
  - Several phases have more than one deliverable.
  - If we need to show a finer level of detail to assist in the monitoring and control of the project, phases can be split so that each sub-phase has only one deliverable

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Project Life Cycles

- The Waterfall Lifecycle Model
  - The waterfall model has several several criticisms:
    - ✓ Real projects rarely follow such a simple sequential lifecycle. Project phases overlap and activities may have to be repeated
    - ✓ It is almost inevitable that some tasks will have to be repeated. The cyclical repetition of tasks is termed iteration.
    - ✓ It tends to be unresponsive to changes in client requirements or technology during the project.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Project Life Cycles

- The Waterfall Lifecycle Model
  - The waterfall model has several several criticisms:
    - ✓ A great deal of time may elapse between the initial systems engineering and the final installation. Requirements will almost inevitably have changed in the meantime and users find little use in a system that satisfies yesterday's requirements but hampers current operations

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

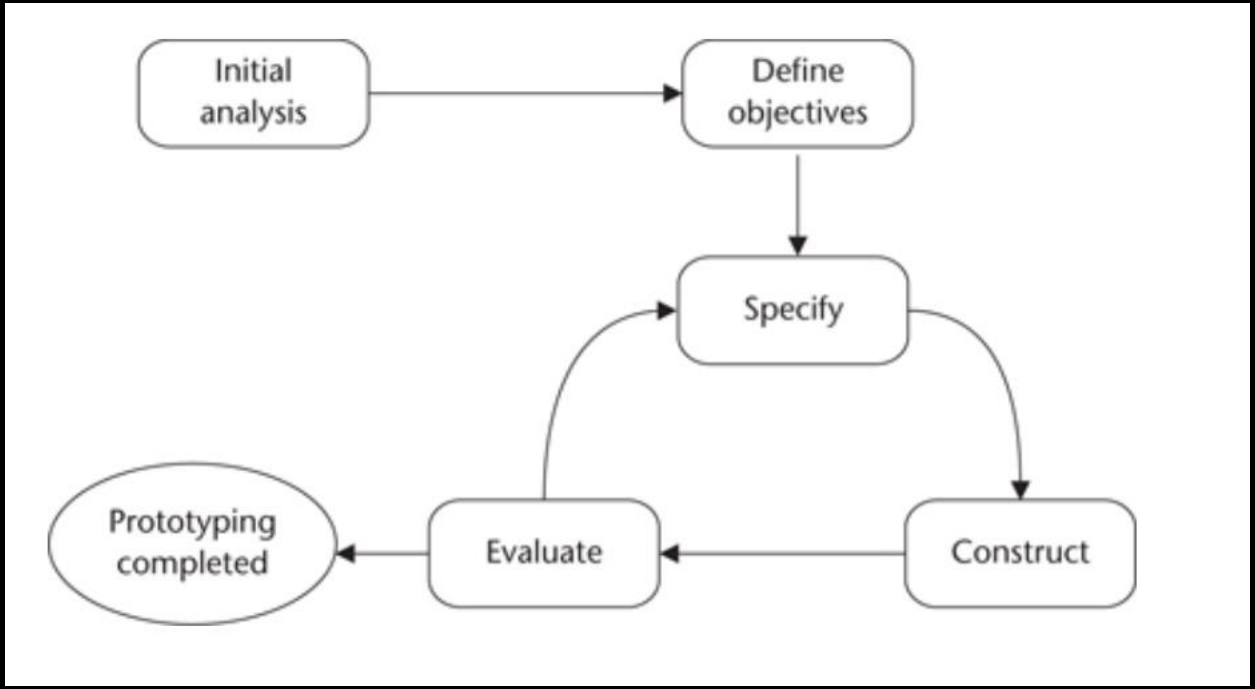
Project Life Cycles

- Prototyping
  - The prototyping approach overcomes many of the potential misunderstandings and ambiguities that may exist in the requirements.
  - In software development a prototype is a system or a partially complete system that is built quickly to explore some aspect of the system requirements and that is not intended as the final working system.
  - A prototype system is differentiated from the final production system by incompleteness and perhaps by a less-resilient construction

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Project Life Cycles

- Prototyping



ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Project Life Cycles

- Prototyping
  - The main stages required to prepare a prototype are:
    - ✓ Perform an initial analysis : to determine the general requirements of the software so that the particular aspects that should be prototyped can be identified.
    - ✓ Define prototype objectives.
    - ✓ Specify prototype.
    - ✓ Construct prototype.
    - ✓ Evaluate prototype and recommend changes.
    - ✓ If the prototype is not completed, repeat the process from the specify prototype stage

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Project Life Cycles

- Prototyping
  - Prototyping has the following advantages:
    - ✓ Early demonstrations of system functionality help identify any misunderstandings between developer and client.
    - ✓ Client requirements that have been missed are identified.
    - ✓ Difficulties in the interface can be identified.
    - ✓ The feasibility and usefulness of the system can be tested, even though, by its very nature, the prototype is incomplete

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Project Life Cycles

- Prototyping
  - Prototyping also has several problems :
    - ✓ The client may perceive the prototype as part of the final system, may not understand the effort that will be required to produce a working production system and may expect delivery soon.
    - ✓ The prototype may divert attention from functional to solely interface issues.
    - ✓ Prototyping requires significant user involvement, which may not be available.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Project Life Cycles

- Iterative and incremental development
  - made up of a series of development activities that are repeated
  - Each of these repetitions is an iteration and each can be viewed as a mini-project in its own right producing new artifacts or successively better or more complete artifacts

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

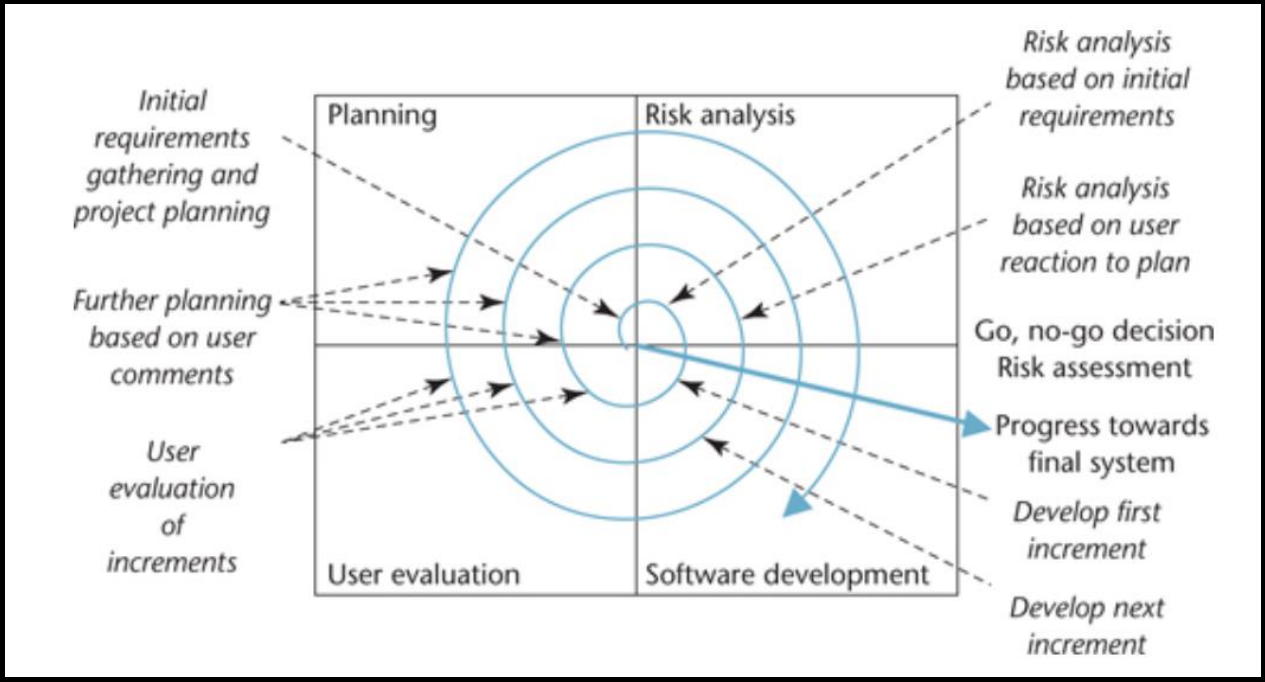
Project Life Cycles

- Iterative and incremental development
  - An incremental approach performs some initial analysis to scope the problem and identify major requirements.
  - Those requirements that will deliver most benefit to the client are selected to be the focus of a first increment of development and delivery.
  - The installation of each increment provides feedback to the development team and informs the development of subsequent increments
  - The spiral model can be viewed as supporting incremental delivery

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Project Life Cycles

- Iterative and incremental development
  - The figure below shows how the spiral model can be adapted to suit incremental delivery.



ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Methodological Approaches

- A methodology comprises an approach to software development, a set of techniques and notations that support the approach, a lifecycle model to structure the development process and a unifying set of procedures and philosophy.
  - ✓ E.g., RUP is an object-oriented methodology that uses UML and follows an iterative and incremental lifecycle.

- Adopting an appropriate methodology for a software development project is one of the most important factors in minimizing the problems in software development

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

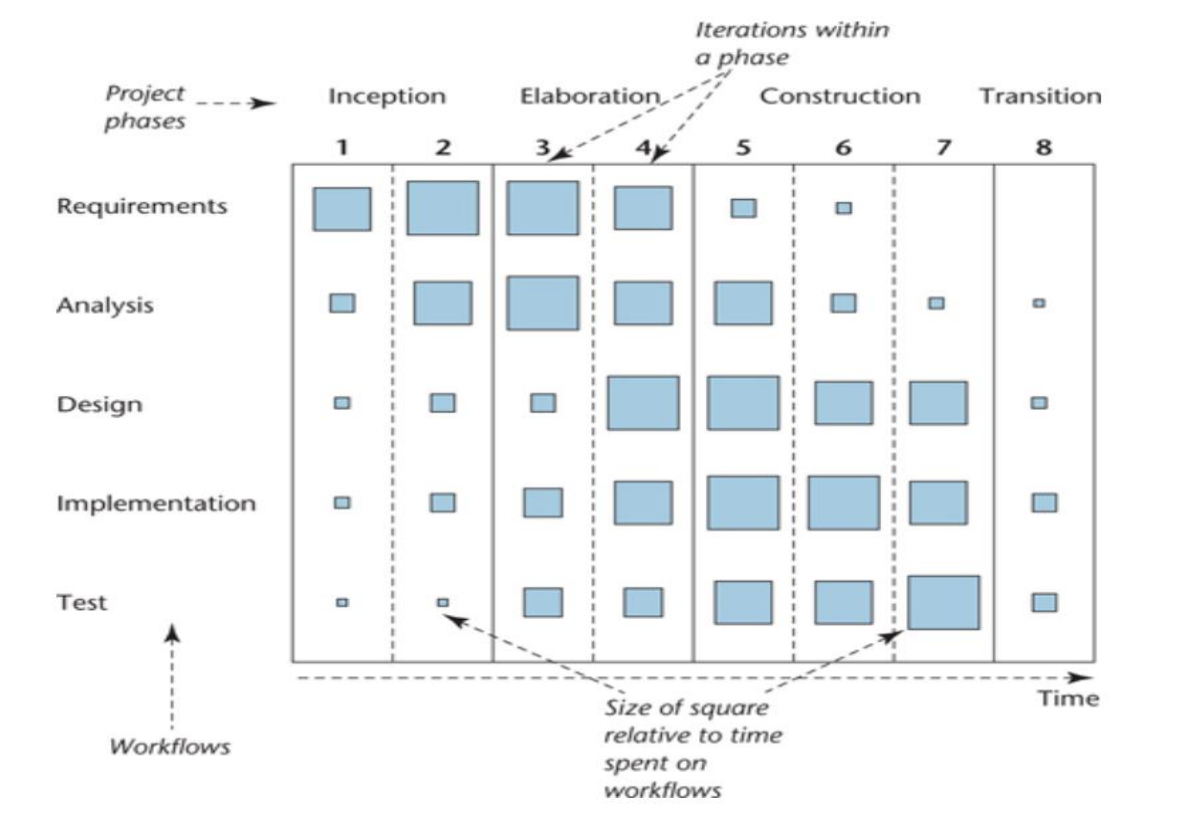
Methodological Approaches

- Unified Software Development Process (USDP)
  - Comprises of four phases.
    - ✓ Inception is concerned with determining the scope and purpose of the project.
    - ✓ Elaboration focuses on requirements capture and determining the structure of the system.
    - ✓ Construction's main aim is to build the software system.
    - ✓ Transition deals with product installation and rollout.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Methodological Approaches

- Unified Software Development Process (USDP)



ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Methodological Approaches

- Unified Software Development Process (USDP)
  - ✓ The IBM Rational Unified Process (RUP) incorporates much of the practice embodied in USPD and has been developed significantly beyond the specification of USDP
  - ✓ RUP has the same phases as USDP but has a more extensive series of workflows or activities (they are called disciplines in RUP).

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Methodological Approaches

- Unified Software Development Process (USDP)
  - The phases of RUP are:
    - **Business modelling** focuses on understanding the business, its current problems and areas for possible improvement.
    - **Requirements** describes how to identify and document user requirements.
    - **Analysis and design** is concerned with building analysis and design models from which the system can be constructed.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Methodological Approaches

- Unified Software Development Process (USDP)
  - The phases of RUP are:
    - **Implementation** deals with the coding and construction of the system.
    - **Test** verifies that the products developed satisfy the requirements identified.
    - **Deployment** deals with product releases and the delivery of the software to the end users

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Methodological Approaches

- Agile approaches
  - A group of software developers and methodology authors met in February 2001 and produced a manifesto for Agile software development.
  - Their objective was to introduce approaches to software development that are less bureaucratic, less focused on documentation and more focused on user interaction and the early delivery of working software than current heavyweight methodologies.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Methodological Approaches

- Agile approaches
  - Their manifesto is shown below:

Manifesto for Agile Software Development

We are uncovering better ways of developing software
by doing and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Methodological Approaches

- Agile approaches
  - Problems with some of the methodologies in the 1980s and 1990s that incorporated the waterfall lifecycle included unresponsiveness to change and a highly bureaucratic approach to analysis and design that was also heavy on documentation
  - In order to overcome some of these problems, iterative lightweight approaches emerged that were typically used for small to medium business information systems where there was appreciable requirements change during the life of the project

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Methodological Approaches

- Agile approaches
  - A fundamental feature of Agile approaches is the acceptance that user requirements will change during development and that this must be accommodated by the development process
  - Agile development is a project management methodology that values individuals and interactions over processes and tools

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

What is object-orientation?

- Object-orientation is an approach to systems development that helps to avoid many of the problems and pitfalls described earlier
- In an object-oriented program, data is encapsulated (or bundled together) with the functions that act upon it.
- This is fundamentally different from most of the earlier approaches to program organization, which typically stressed the separation of data and functions.
- An object is also a conceptual unit of both analysis and design, thus the same conceptual unit links analysis to design and implementation.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

What is object-orientation?

- This, more than anything else, is what makes it possible for object-oriented projects to follow an iterative lifecycle.
- Apart from the object itself, the most important concepts are :
  - Class
  - Instance
  - generalization and specialization
  - Encapsulation
  - information hiding
  - message passing
  - polymorphism.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Objects concepts

- Object
  - ✓ An abstraction of something in a problem domain, reflecting the capabilities of the system to keep information about it, interact with it, or both.
  - ✓ Abstraction in this context means a form of representation that includes only what is important or interesting from a particular viewpoint

  - ✓ An object 'has state, behaviour and identity

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Objects concepts

- Object
  - ✓ 'identity' means simply that every object is unique, while 'state' and 'behaviour' are closely related to each other.
  - ✓ 'State' represents the condition that an object is in at a given moment, in the sense that an object's state determines which behaviours, or actions, an object can carry out in response to a given event
  - ✓ For a software object, 'state' is the sum total of the values of the object's data while 'behaviour' stands for the ways that an object can act in response to events

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Objects concepts

- Object
  - Some objects characteristics

Object	Identity	Behaviour	States
A person	'Hussain Pervez'	Speak, walk, read	Studying, resting, qualified
A shirt	'My favourite button-down white denim shirt'	Shrink, stain, rip	Pressed, dirty, worn
A sale	'Sale no. 0015, 15/02/10'	Earn loyalty points	Invoiced, cancelled
A bottle of ketchup	'*This* bottle of ketchup'	Spill in transit	Unsold, opened, empty

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Objects concepts

- Class and object
  - ✓ Class is a concept that describes a set of objects that are specified in the same way.
  - ✓ Here, we mean objects as abstractions within an information system—either a model or the resulting software—not the real-world objects that they represent.
  - ✓ All objects of a given class share a common specification for their features, their semantics and the constraints upon them

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

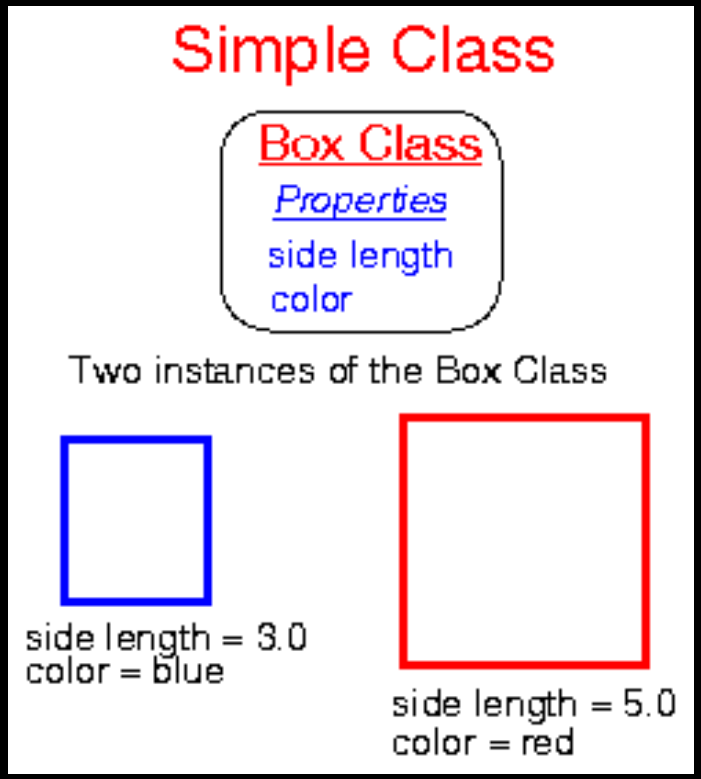
Objects concepts

- Class and object
  - ✓ This does not quite mean that all objects of a class are identical in every way, but it does mean that their specification is identical
  - ✓ Objects that are sufficiently similar to each other are said to belong to the same class
  - ✓ A single object is also known as an instance. This carries a connotation of the class to which the object belongs.
  - ✓ Every object is an instance of some class.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Objects concepts

- Class and object



ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

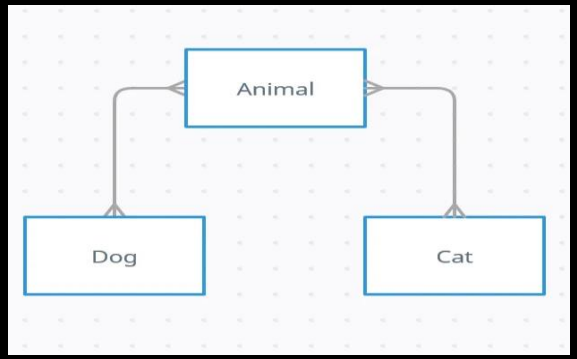
Objects concepts

- Generalization and specialization
  - ✓ Generalization and specialization are very important to object-orientation.
  - ✓ They help programmers, designers and analysts to reuse previous work instead of repeating it
  - ✓ They also help to organize complex models and systems in a way that makes them more understandable.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Objects concepts

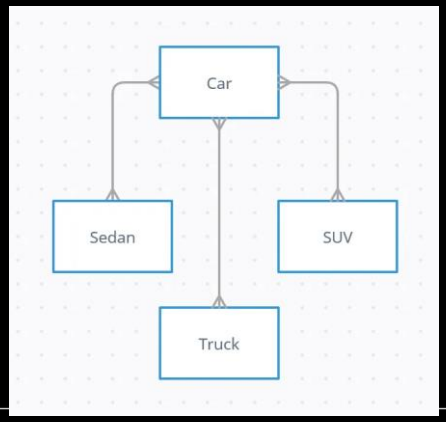
- Generalization and specialization
  - ✓ Specialization is the process of creating a new class that inherits from an existing class.
  - ✓ The new class, also known as a subclass, inherits all of the properties and methods of the parent class, but can also add new properties and methods or override the existing ones.



ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Objects concepts

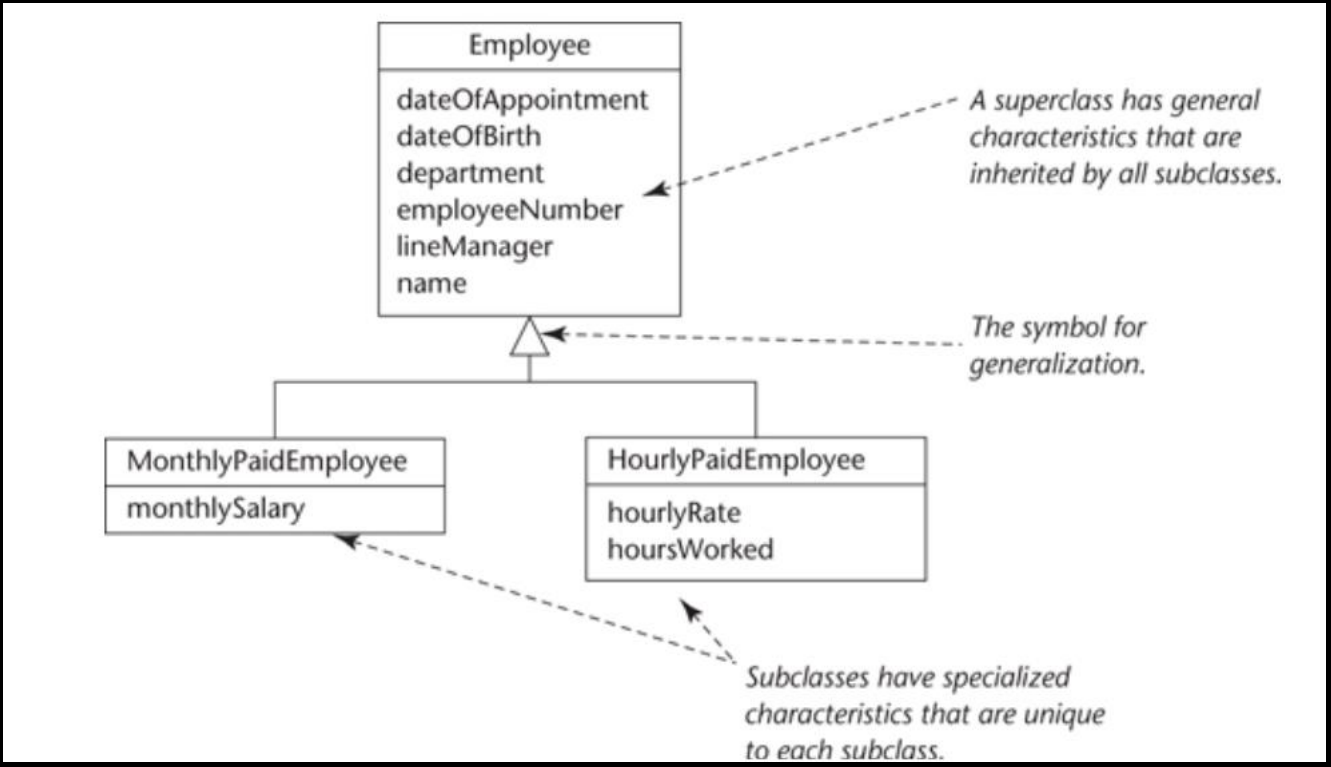
- Generalization and specialization
  - ✓ Generalization is the opposite of specialization.
  - ✓ It is the process of creating a more general class that can be used as a base for other more specific classes.
  - ✓ In other words, generalization allows us to create a superclass that has common properties and methods shared by multiple subclasses.



ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Objects concepts

- Generalization and specialization



ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Objects concepts

- Encapsulation, information hiding and message passing
  - ✓ These three concepts are closely related, so we will consider them together.
  - ✓ Encapsulation is a feature of object-oriented programming, which is also applied in analysis modelling.
    - ➢ It means the placing of data within an object together with the operations that apply to that data.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Objects concepts

- Encapsulation, information hiding and message passing
  - An object really is little more than a bundle of data together with some processes that act on the data.
  - The data is stored within the object's attributes; together these comprise the object's information structure

- Information hiding goes one step further than encapsulation, and ideally makes it impossible for one object to access another object's data in any other way than through calls to its operations.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

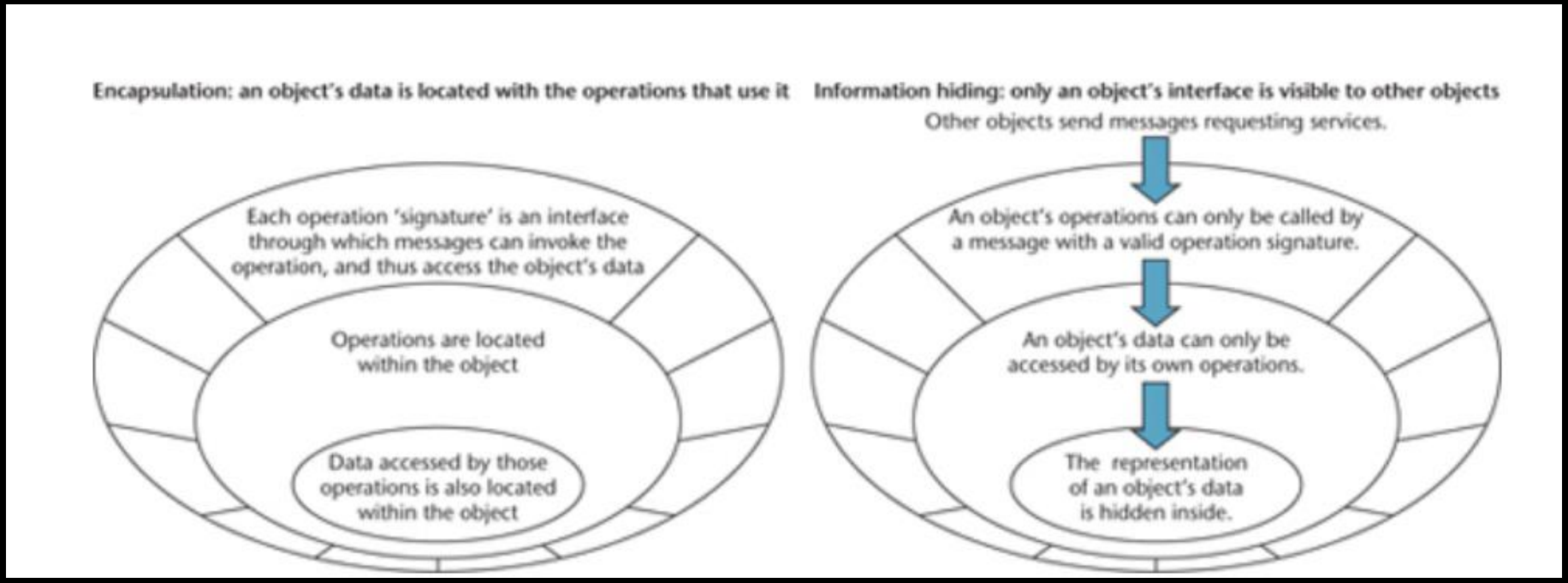
Objects concepts

- Encapsulation, information hiding and message passing
  - ✓ This insulates each object from the need to 'know' any of the internal details of other objects.
  - ✓ Essentially, an object only needs to know its own data and its own operations.
  - ✓ However, many processes are complex and require collaboration between objects.
  - ✓ The 'knowledge' of some objects must therefore include knowing how to request services from other object

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Objects concepts

- Encapsulation, information hiding and message passing



ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

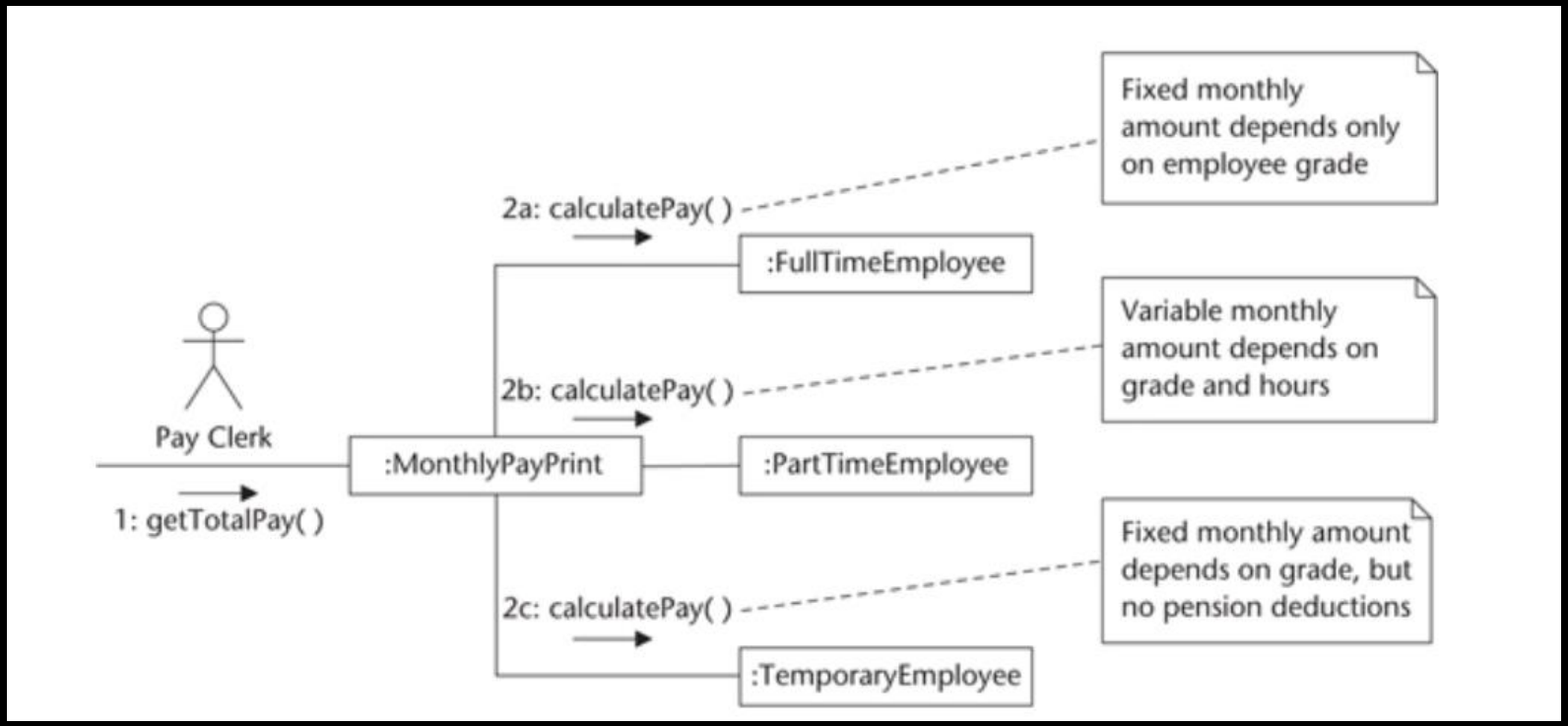
Objects concepts

- Polymorphism
  - Polymorphism literally means 'an ability to appear as many forms' and it refers to the possibility of identical messages being sent to objects of different classes, each of which responds to the message in a different, yet still appropriate, way
  - This means the originating object does not need to know which class is going to receive the message on any particular occasion.
  - The key to this is that each object knows how to respond to valid messages that it receives

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Objects concepts

- Polymorphism



ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Objects concepts

- Object state
  - ✓ An object's state is defined as the totality of the current values of data within the object and its associations with other objects.
  - ✓ All states are not equally important, but some differences of state imply significant differences in the behaviour that the object will display in response to the same message
  - ✓ In the real world, people and objects do not always behave in the same way in response to similar stimuli.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

UML Background

- The Unified Modeling Language (UML) is a graphical language for visualizing, specifying, constructing, and documenting the artifacts of a software-intensive system.

- The UML offers a standard way to write a system's blueprints, including conceptual things such as business processes and system functions as well as concrete things such as programming language statements, database schemas, and reusable software components

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

UML Background

- UML is a language (Unified Modeling Language) for models
  - Used for technical and graphical specification
  - It is Graphic notation to visualize models
  - It is Not a method or procedure
- The UML is a language for
  - ✓ Visualizing
  - ✓ Specifying
  - ✓ Constructing
  - ✓ Documenting

  the artifacts of a software-intensive system.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Models and Diagrams

- A model is an abstract representation of something real or imaginary.
- They are useful in several different ways, precisely because they differ from the things that they represent.
  - ✓ A model is quicker and easier to build.
  - ✓ A model can be used in simulations, to learn more about the thing it represents.
  - ✓ A model can evolve as we learn more about a task or problem.
  - ✓ We can choose which details to represent in a model, and which to ignore.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Models and Diagrams

- ✓ A model is an abstraction.
- ✓ A model can represent real or imaginary things from any domain
- ✓ Many different kinds of thing can be modelled.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

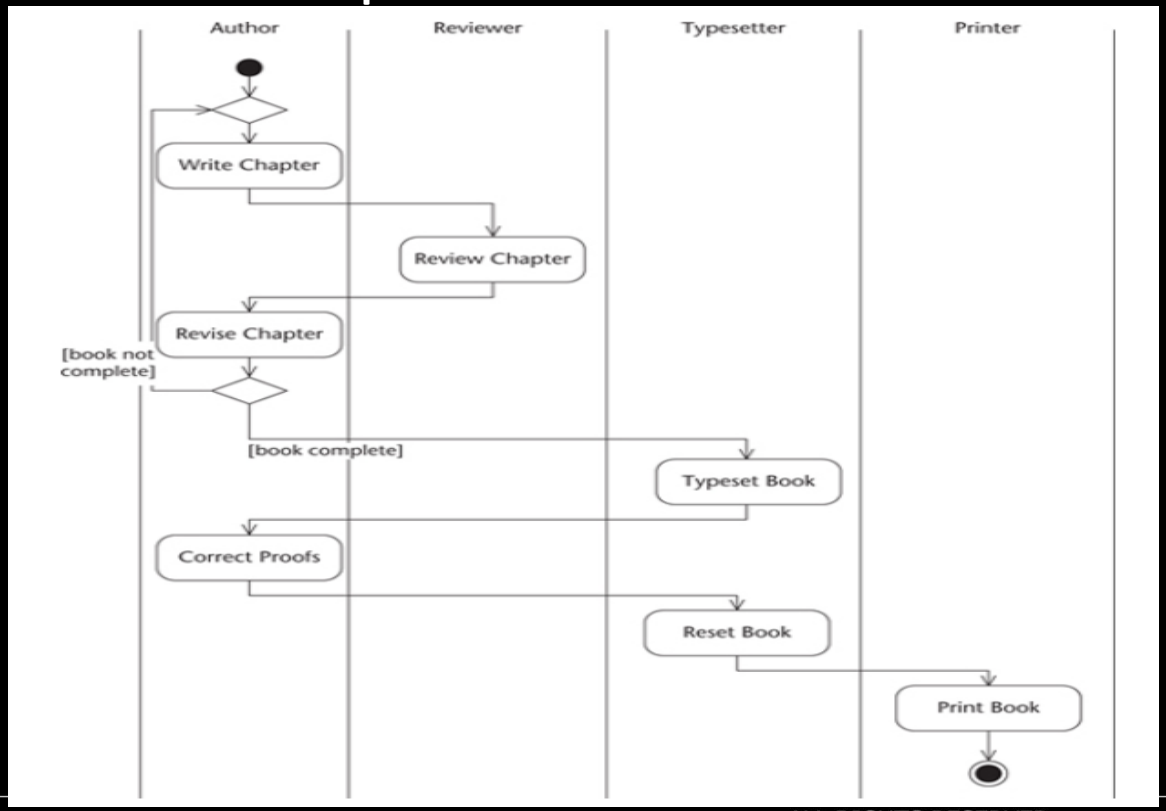
Models and Diagrams

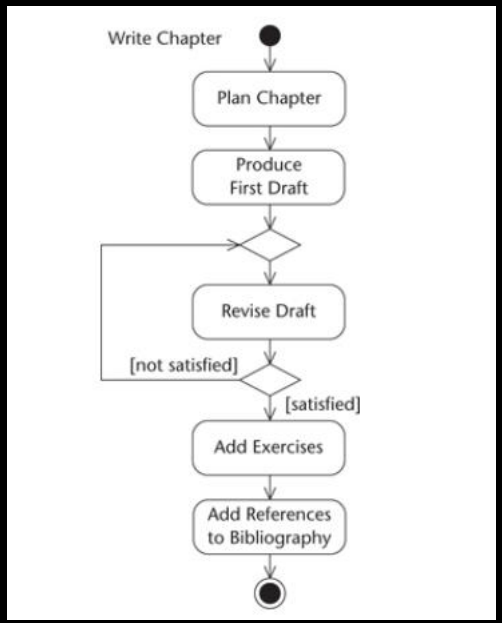
- A diagram is a visual representation of some part of a model.
- Analysts and designers use diagrams to illustrate models of systems in the same way as architects use drawings and diagrams to model buildings.
- Diagrammatic models are used extensively by systems analysts and designers in order to:
  - ✓ communicate ideas
  - ✓ generate new ideas and possibilities
  - ✓ test ideas and make predictions
  - ✓ understand structures and relationships

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

Models and Diagrams

- An activity Diagram for producing a book and one for Write Chapter





ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

MODELS IN UML

- In UML there are a number of concepts that are used to describe systems and the ways in which they can be broken down and modelled.
- A system is the overall thing that is being modelled.
- A subsystem is a part of a system, consisting of related elements.
- A model is an abstraction of a system or subsystem from a particular perspective or view

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

MODELS IN UML

Figure 5.5 UML notation for packages, subsystems and models.

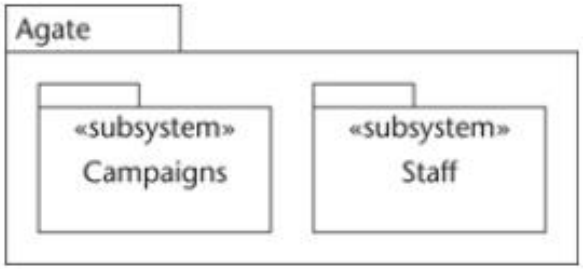


Figure 5.6 UML notation for a system containing subsystems, shown by containment.

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

UML Tools

- Visio
- Dia
- Lucidcharts
- StarUML

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini

THE END

ALL RIGHTS RESERVED
No part of this document may be reproduced without written approval from Limkokwing University Of Creative Technology Eswatini